

Arcora — Litepaper

Arcora — Litepaper

Version 0.3 · 2026-05-20

Stablecoin checkout on Arc Network. The customer pays with one signature in any supported stable. The merchant settles in the stable they want, deterministically. No bridge clicks, no custodial off-ramp, no FX gymnastics.

Contents

1. [The bet, in one paragraph](#)
 2. [Why this exists](#)
 3. [System at a glance](#)
 4. [How a payment settles](#)
 5. [Custody escrow — the design](#)
 6. [Off-chain trust boundaries](#)
 7. [Compliance posture](#)
 8. [Developer surface](#)
 9. [Live deployment](#)
 10. [Roadmap](#)
 11. [What stands between us and mainnet](#)
 12. [Glossary](#)
-

1. The bet, in one paragraph

A merchant on Arc says: "I want to be paid in EURC." A customer holds USDC. Arcora's gateway accepts the customer's pay-in token, runs an atomic on-chain swap, and deposits the merchant's chosen stable into a 7-day custody escrow that the merchant claims permissionlessly. The customer signs once — a Permit2 message, no transaction, no gas. The merchant integrates once — an API key, an invoice call, a webhook secret. Everything between those two surfaces (FX routing, key isolation, settlement, refund window, compliance gating) is the protocol.

The roadmap extends this to **cross-chain**: customer pays from any chain Arc App Kit Bridge supports, merchant still settles in their preferred Arc stable. That's v2.0, spec'd but not built. The current product (v1.2) ships the Arc-side version of the same flow.

2. Why this exists

Stablecoins moved roughly \$5 trillion in 2025; USDC alone accounts for most of it. The holder-to-holder UX is fine — wallet to wallet, single chain. The merchant UX is not:

- The customer's chain is not the merchant's chain.
- The customer's token is not the merchant's token.
- Bridges, DEXes, approval clicks. Each one is a place the customer abandons.
- The fallback is a custodial off-ramp that charges 1–2% and takes days.

Asking every merchant to “just hold any stable and figure out FX later” pushes treasury complexity onto every merchant. That's not a product. That's a tax.

Arcora collapses the merchant integration to a single API call and the customer's path to a single signature.

3. System at a glance

Four moving parts. The contract is intentionally the smallest of them.



- **The app** runs the merchant dashboard, hosted checkout, and the HTTP API. Stateless Vercel functions, Iron Session for cookies.
- **The contract** is the invoice lifecycle machine plus the custody escrow plus the protocol-fee accumulator. It does not swap and does not orchestrate.
- **The relayer** is the only off-chain actor with `RELAYER_ROLE`. It calls Permit2, runs the swap via Circle's App Kit, and submits `settleInvoice` to the contract.
- **The indexer and webhook daemons** are passive: they read on-chain events, write to Postgres, and HMAC-sign outbound webhooks.

The architecture is small on purpose. A merchant only needs to think about: API call, webhook secret, escrow claim. Everything else is the protocol's job.

4. How a payment settles

A USDC → EURC payment end-to-end. Times are typical observed latencies; nothing is a guarantee.

```
T+0    Merchant creates invoice
      POST /api/invoices (X-Arcora-API-Key)
      → server-side compliance screen on the merchant payout wallet
      → server calls createInvoiceFor on the gateway (server hot wallet)
      → returns { invoiceId, url }

T+5s   Customer opens checkout
      GET /i/<invoiceId>
      → renders QuoteDisplay, ConnectButton, PayButton

T+10s  Customer connects wallet
      → /api/checkout/authorize compliance-screens the customer wallet
      → button transitions to "Pay with one signature"

T+15s  Customer signs Permit2 EIP-712
      → wallet shows typed data; witness binds the signature to
        (invoiceId, relayer)
      → POST /api/checkout/submit enqueues the row for the relayer
      → no transaction, no native gas

T+20s  Relayer claims the row (SELECT FOR UPDATE SKIP LOCKED)
      → calls Permit2.permitWitnessTransferFrom – pulls USDC
      → runs Circle App Kit Swap → maker network fills, EURC arrives
      → if filled amount < amountOut: off-chain refund + recordPayerRefund
      → else: transfer EURC to the gateway, call settleInvoice

T+25s  Gateway settles
      → escrow[globalId] populated with the merchant's amountOut
      → InvoicePaid + SettlementContext + EscrowCreated events

T+27s  Indexer reconciles
      → UPDATE invoices SET status='paid', paid_at = block.timestamp
      → enqueues invoice.paid webhook

T+30s  Webhook daemon delivers
      → HMAC-SHA256 signs the body, POSTs to merchant webhookUrl
      → SSRF guard re-resolves DNS at delivery time

T+7d   Merchant claims
      → calls gateway.claim([globalId, ...])
      → escrow drains into merchants[merchant].payoutAddress
```

Refund within the window:

```
M Merchant or refund-delegate calls gateway.refundInvoice(globalId)
  → escrow.amount returns to the customer's payer wallet
  → invoice status → Refunded
```

Failed swap (off-chain refund path):

```
S Relayer sees swap fill < inv.amountOut
  → ERC20.transfer the USDC pulled-via-Permit2 back to the customer
  → gateway.recordPayerRefund(globalId, payer, USDC, owedBack, reasonHash)
  → invoice status → Failed (no swap happened on-chain, no fee charged)
```

The customer is whole at every step. The merchant is paid only when the contract holds enough payout token. The protocol is solvent because every fee accrual happens at settlement time against funds the contract already holds.

5. Custody escrow — the design

The contract is not a swap router and not a settlement bridge. It is an invoice lifecycle machine wrapped around a fee accumulator. The interesting design choices:

Settlement deposits to escrow, not to the merchant. `settleInvoice` transfers the gross payout into `escrows[globalId]`. The merchant cannot front-run a refund because there is no transfer yet. The customer is protected for the entire `REFUND_WINDOW` (7 days). After that, the merchant calls `claim()` — permissionless, batched, deterministic.

`recordPayerRefund` is the relayer's safe-failure path. If App Kit Swap fills below the merchant's floor, the relayer never calls `settleInvoice`. It returns the pay-in token off-chain and emits an event that flips the invoice to `Failed`. The customer is whole. The merchant gets an `invoice.failed` webhook. No on-chain state moved into an inconsistent place.

`payInToken` is validated, not trusted. The relayer reports which token the customer actually paid in. The contract checks it against what the invoice was created with. A buggy or hostile relayer cannot lie in the on-chain log about the source token.

The protocol fee is the excess. When App Kit returns a rate-favourable fill, the surplus over `amountOut` accrues to `protocolFeesAccrued[token]`. The `InvoicePaid` event surfaces this amount so downstream accounting sees exactly what the contract banked, not a hardcoded zero.

`adminRecoverEscrow` covers abandoned merchants. After `REFUND_WINDOW + ADMIN_RECOVERY_DELAY` (14 days) and merchant deactivation, an admin can sweep funds to a recovery wallet. The batch is atomic — one invalid id reverts the whole call. No half-recoveries.

Every fund-moving path carries `nonReentrant`, including `withdrawFees`. CEI ordering already blocks today's drain vectors; the modifier is the invariant.

The on-chain surface is small enough to read in one sitting. That's deliberate — the harder a contract is to reason about, the more places things hide.

6. Off-chain trust boundaries

The relayer is the only off-chain component that touches custody-class authority. Everything else is reconciliation.

Key isolation. The relayer's private key lives in HashiCorp Vault's KV-v2 store, encrypted at rest with Vault's master key. The daemon AppRole-logs in once at boot, fetches the key into process memory, and uses it to sign settlement transactions and the App Kit swap adapter context. The key never lands in any `.env` file on disk.

AppRole rotation. The Vault `role_id` is long-lived; the `secret_id` rotates daily, well before its 24-hour TTL. The rotation cron uses a least-privilege operator token whose only grant is `update auth/approle/role/relayer/secret-id`. Not the root token.

The relayer can settle. It cannot withdraw fees, change roles, or pause. `RELAYER_ROLE` and `DEFAULT_ADMIN_ROLE` are separate. A relayer compromise costs the in-flight Permit2 transfers — capped by the customer's signed `amountIn` per invoice — and does not touch escrowed funds or accrued protocol fees.

The server hot wallet pays invoice-creation gas. Its private key is AES-256-GCM encrypted at rest (32-byte master key from env, 12-byte IV per record, auth tag). The decryption key (`MASTER_KEY`) is the smallest single point of failure — protected by environment isolation and rotation discipline.

API keys are bcrypt-prefixed. Lookups hit a searchable prefix; the hash verifies. The plaintext key is revealed once at issue and never re-fetched.

Webhook secrets are AES-256-GCM at rest. Verified at delivery, not at registration, with an SSRF guard that re-resolves the destination DNS each time. RFC1918 / loopback / link-local / CGN / IPv6-mapped IPv4 / cloud metadata addresses are rejected.

SIWE for the merchant dashboard. Domain bind, chain bind, atomic nonce consume — a signature collected on another domain or for another chain cannot authenticate here.

Redirect URLs are triple-checked. Merchant-declared allowlist at invoice creation, server-side SSRF re-resolve, client-side safeRedirect at navigation. Open-redirect surface is closed at every layer.

7. Compliance posture

Compliance is wired in as a config-flip surface, not a contract feature. The default provider on testnet is `Noop` — calls succeed, every wallet `allow` s. Real providers (Elliptic, TRM Labs) are scaffolded behind an env flag with the adapter shape already in code and tested.

Two screening flows:

- **Merchant payout** at invoice creation. The on-chain payout address is read from the gateway and screened — not the merchant's identity wallet. A merchant who rotates to an unscreened wallet hits the gate before a new invoice routes to it.
- **Customer pay** at `/api/checkout/authorize` , before the Permit2 signature is requested. If the wallet is `reject` 'd, the button stays disabled with explanation. If `review` 'd, the customer sees a clear "wallet can't be used for this payment" panel with a ticket id.

Results are cached per address by provider TTL: sanctions hits live seven years, other categories thirteen months. Indexed for cron-driven retention sweep.

Fail-closed for customer screening (a provider outage shouldn't pass a wallet through). Fail-open for invoice creation (a provider outage shouldn't block legitimate merchants from minting invoices). Each is an explicit operator flag.

KYB is spec'd separately (`ManualKybProvider` for testnet, `PersonaProvider` for post-revenue) and is mainnet-gated. The current testnet posture is wallet-only.

8. Developer surface

A merchant integrates in four touch points. The full reference lives at arcorapay.xyz/docs ; what follows is the shape.

SDK. `@arcora/sdk` (TypeScript, ESM + CJS + IIFE for `<script>`). The instance API is the canonical path; the singleton `Arcora.init` is `@deprecated` .

```
import { Arcora } from "@arcora/sdk";

const arcora = new Arcora({ apiKey, environment: "testnet" });

const inv = await arcora.createInvoice({
  amountUsdc: 49.99,
  payInToken: "EURC",
  successUrl: "https://yoursite.com/orders/done",
});

arcora.openCheckout(inv);
```

React SDK. `@arcora/sdk-react` wraps the same API for hook + component use:

```
import { useCheckout, CheckoutButton } from "@arcora/sdk-react";

const { checkout, loading, error, refundEndsAt } = useCheckout({ apiKey });

<CheckoutButton
  apiKey={publicKey}
  invoice={{ amountUsdc: 4.50, payInToken: "EURC", successUrl: "..." }}
>
  Pay €4.50
</CheckoutButton>
```

HTTP API. For server-to-server flows that don't run JavaScript:

Endpoint	Method	Auth	Purpose
<code>/api/invoices</code>	POST	X-Arcora-API-Key	Create invoice
<code>/api/invoices/[id]</code>	GET	X-Arcora-API-Key	Read invoice; full record for owning merchant
<code>/api/merchant/escrows</code>	GET	iron-session	Escrow buckets: pending / matured / claimed
<code>/api/merchant/treasury</code>	GET	iron-session	Per-stable rollups + activity feed

Webhooks. Deliveries to the merchant's `webhookUrl` with HMAC-SHA256 over the raw body:

```
POST https://merchant.example.com/webhooks/arcora
Content-Type: application/json
X-Arcora-Signature: sha256=<hex>

{ "type": "invoice.paid",
  "invoice_id": "0x...",
  "payer": "0x...",
  "tx_hash": "0x...",
  "occurred_at": "2026-05-13T12:06:56Z" }
```

Verification is three lines of `crypto.createHmac('sha256', secret).update(rawBody)` with a constant-time compare. A WordPress plugin (`arcora-woocommerce`) ships in-tree; Shopify is on the roadmap.

9. Live deployment

Arc testnet (chain id 5042002, RPC `https://rpc.testnet.arc.network`, explorer `https://testnet.arcscan.app`):

Component	Address
ArcFXGatewayV11	<code>0x07BAC123A682D24d3eC439ce454cA8AC64eAe3A3</code> (current)
Permit2	<code>0x000000000022D473030F116dDEE9F6B43aC78BA3</code> (Uniswap canonical)
USDC	<code>0x3600000000000000000000000000000000000000</code> (6 decimals)
EURC	<code>0x89B50855Aa3bE2F677cD6303Cec089B5F319D72a</code> (6 decimals)
Gateway owner	<code>0xe8E5AAa3d8c705A07de02aADF98CE31F20A5754b</code>
Relayer (Vault-backed)	<code>0x29EcFedDF31E4dA4a62b89bADe35b224cE144DAE</code>

Gateway parameters: protocol fee 30 bps, refund window 7 days, admin recovery delay 7 days. USDC + EURC pre-whitelisted.

Hosting. The app runs on Vercel (Next.js App Router, Turbopack). Database is Neon Postgres in EU-Central. The three off-chain daemons (relayer, indexer, webhook) run on a single Ubuntu host with HashiCorp Vault on the same machine, listener bound to loopback.

Source. Open development on GitHub: `Kubudak90/arc-fx-gateway`. Default branch `plan-1-protocol`. Tagged releases: `v1.0.0`, `v1.0.1`, `v1.0.2`, `v1.0.3`, `v1.1.0`, `v1.2.0` (current).

10. Roadmap

The intended end-state is **intent-based cross-chain**: customer signs one intent, Arcora solves the full route, merchant settles in their chosen Arc stable. Each milestone below is a step toward that without rewriting earlier layers.

v1.0 — Arc-only checkout (shipped). USDC and EURC, Permit2-based settlement, hosted checkout, merchant dashboard, refunds, treasury reporting, SDK + React SDK + WooCommerce plugin published to npm.

v1.1 — Custody escrow (shipped). Custody-escrow gateway, refund safety net, admin recovery for abandoned merchants, compliance gate (Phase 0), Vault-backed key isolation.

v1.2 — Production hardening (shipped, current). External review + full code audit closed: per-IP rate limiting on checkout endpoints, constant-time secret comparison on the cron auth path, server-side Permit2 signature verification with unit coverage, SSRF + https-only guard on merchant origins, invoice input bounds, configurable compliance asset, working ESLint pipeline, checkout-countdown accessibility, source-tree V11 relabel and legacy cleanup.

v1.x — In-flight enhancements. - Multi-stable (USDT, PYUSD, DAI, USDe) — mainnet-bound, testnet App Kit currently USDC/EURC only. - Shopify plugin — same shape as the WooCommerce one.

v2.0 — Cross-chain USDC source (spec'd). Customer pays USDC from any CCTP-supported EVM chain (Ethereum, Arbitrum, Optimism, Base, Polygon, Avalanche, Linea, Codex). Routes through Arc App Kit Bridge — not raw CCTP — so the bridge surface stays in Circle's supported primitives. Merchant still settles in their chosen Arc stable.

v2.1 — Source-side aggregator. Customer pays in any token on the source chain (native ETH, any ERC20), not only USDC. Odos / 1inch / 0x / Paraswap router on each source chain; reverse route engine computes max-in given target output and slippage.

v2.2 — Non-EVM sources. Solana, Sui — same product surface, different wallet stack.

v3.0 — Intent / solver model. True one-signature one-click: the customer signs an intent, Arcora's solver executes the full route. Multi-month research effort comparing ERC-7683, Across, DeBridge-Liquid. Not started until v2.0 and v2.1 are stable.

11. What stands between us and mainnet

Concrete items, none of them hypothetical. Each has a defined trigger.

1. **Arc Network mainnet launch.** Out of our control — Arc is on testnet, so we are too.
2. **External audit.** Spearbit / Cantina / Sherlock RFP. Trigger: first paying merchant **or** funding round close.
3. **Multisig admin migration.** `DEFAULT_ADMIN_ROLE` is a single EOA today; 2-of-3 or 3-of-5 at T-0.
4. **KYB go-live.** `ManualKybProvider` (testnet, \$0) + `PersonaProvider` (post-revenue) wired into the merchant signup flow.
5. **Vault hardening.** TLS on the listener (cert + dedicated host), dedicated Unix user for relayer/indexer/webhooks, eventual migration of the relayer key to an HSM-isolated signer.
6. **Real Chainlink price feeds.** Replace mock feeds per stable; oracle keepalive timer becomes obsolete.
7. **Compliance provider activation.** Flip `COMPLIANCE_PROVIDER` from `noop` to `elliptic` or `trmlabs`.
8. **Domain canonicalisation.** `arcorapay.xyz` is live; consolidate on the canonical mainnet domain at T-0.
9. **Bug bounty.** Immunefi engagement with first paying merchant.

Items 1–4 and 9 require coordination beyond the protocol. Items 5–8 are reversible single-host or single-config changes.

12. Glossary

- **Permit2** — Uniswap’s canonical EIP-712 transfer authorisation contract at `0x00000000000022D473030F116dDEE9F6B43aC78BA3`. Lets a customer authorise a single transfer without an on-chain `approve`. With a **witness**, the authorisation is scoped to a specific application context — here `(invoiceId, relayer)` — so a signature cannot be replayed against a different invoice or by a different relayer.
- **App Kit Swap** — Circle’s RFQ-based stablecoin swap on Arc. Maker network fills swap requests; integration via the `@circle-fin/app-kit` SDK. Used by Arcora’s relayer for the FX leg.
- **Custody escrow** — The pattern where settled funds sit in the contract (not the merchant’s wallet) until a refund window closes, after which the merchant permissionlessly claims. Arcora uses a 7-day refund window + 7-day admin recovery delay.
- **SIWE** — Sign-In With Ethereum (EIP-4361). Used for merchant dashboard auth, bound by domain + chain id with atomic nonce consume.
- **AppRole** — HashiCorp Vault’s machine-auth method. `role_id` (long-lived) + `secret_id` (rotatable, time-limited) exchange for a Vault token. Used to fetch the relayer key at boot.
- **KV-v2** — Vault’s versioned key-value secrets engine. Reads at path `secret/data/<name>`.
- **CCTP** — Circle’s Cross-Chain Transfer Protocol. Burn-and-mint USDC moves across supported chains; v2.0 source-chain layer for Arcora.

This document captures live state at the date stamped above. Contract addresses, ops topology, and roadmap items change over time — `packages/contracts/deployments/arc-testnet.json` is the canonical record for on-chain state.